



Money Laundering Analysis

Sistemas Distribuidos

108397 Ordoñez Alejo

107863 Minervino Lorenzo

106010 Valsagna Federico

Índice

1. Arquitectura del sistema	3
2. Casos de uso	3
3. DAG de procesamiento	4
3.1. UC1 - Filtro directo	4
3.2. UC2 - Máximo por banco	6
3.3. UC3 - Comparación entre períodos	8
3.4. UC4 - Patrón scatter-gather	10
3.5. UC5 - Conteo con conversión de moneda	12
4. Vista física	14
4.1. Robustez	14
4.1.1. UC1	14
4.1.2. UC2	15
4.1.3. UC3	16
4.1.4. UC4	18
4.1.5. UC5	20
4.1.6. Overview del sistema completo	22
4.2. Despliegue	25
5. Vista de desarrollo	26
5.1. Paquetes	26
6. Vista de procesos	29
6.1. Actividades	29
6.1.1. UC1	29
6.1.2. UC2	30
6.1.3. UC3	31
6.1.4. UC4	32
6.1.5. UC5	32
6.2. Secuencia	33
6.2.1. UC1	33
6.2.2. UC2	33
6.2.3. UC3	34
6.2.4. UC4	34
6.2.5. UC5	35
7. Tolerancia a fallos	36
7.1. Arquitectura de tolerancia a fallos	36
7.2. Detección de caídas	37
7.3. Cluster de supervisores	38
7.4. Fin de stream y barrera de anillo	40
7.5. Checkpointing y deduplicación	41
7.6. Puntos de falla	43
7.6.1. Caída a mitad de batch	43
7.6.2. Caída tras el checkpoint y antes de los acks	43

7.6.3.	Caída de un nodo con estado acumulado	44
7.6.4.	Caída del cliente	45
7.7.	Rendimiento bajo tolerancia a fallos	46
7.7.1.	Frecuencia: matar más seguido	46
7.7.2.	Ráfaga: matar más de golpe	47
7.7.3.	El costo del checkpoint	48
8.	Testing	49
8.1.	Testing del pipeline	49
8.2.	Testing de tolerancia a fallos	49
8.2.1.	Pruebas determinísticas	49
8.2.2.	Demo con Chaos Monkey	49
8.3.	Testing de escalabilidad	50
9.	Desarrollo	51
9.1.	Tareas a realizar	51
9.2.	Asignación	51

1. Arquitectura del sistema

El sistema recibe un conjunto de datos de transacciones bancarias desde uno o más clientes simultáneamente y devuelve los resultados de cinco análisis distintos. La diversidad de los requerimientos, que abarcan desde un filtro simple hasta la detección de patrones en grafos y la consulta a servicios externos, motivó el diseño de una arquitectura de procesamiento distribuido en pipeline, donde cada caso de uso recorre una cadena de nodos especializados de forma concurrente con los demás.

2. Casos de uso

Una petición del cliente hace que el sistema procese los 5 casos de uso:

- **UC1** Cuenta de origen, cuenta de destino y monto para transacciones USD con monto menor a \$50.
- **UC2** Nombre de banco, cuenta de origen y monto de la máxima transacción en USD para cada banco.
- **UC3** Cuenta de origen y monto de transacciones USD en el período 2022-09-06 al 2022-09-15 (período B), cuyo monto sea menor a un centésimo del promedio de monto para su formato en el período 2022-09-01 al 2022-09-05 (período A).
- **UC4** Pares de cuentas que cumplan con el patrón *scatter-gather* con una cuenta de separación y una cantidad mínima de cuentas intermedias igual a 5; en el período A.
- **UC5** Cantidad de transacciones con formato de pago *Wire* o *ACH* en el período A, cuyo monto en USD sea menor a 1.

Los cinco casos de uso presentan niveles crecientes de complejidad: desde un filtro directo (UC1) hasta la detección de estructuras en un grafo de transacciones (UC4) o la normalización de moneda contra un servicio externo (UC5). Esta escalera de complejidad da forma a la arquitectura del sistema y se verá reflejada en cada una de las vistas que siguen.

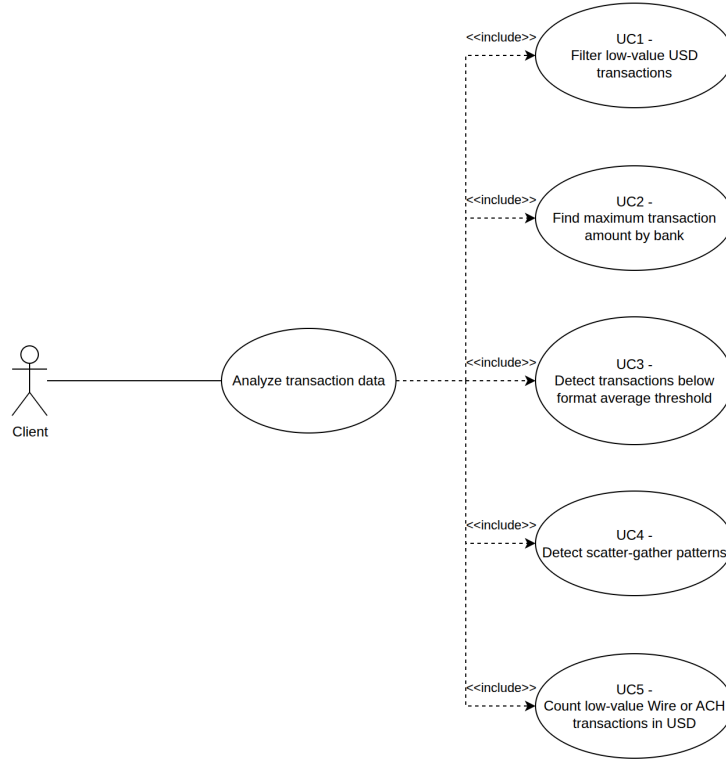


Figura 1: Diagrama de casos de uso

3. DAG de procesamiento

El conjunto de pipelines puede modelarse como un DAG (*Directed Acyclic Graph*). La ausencia de ciclos garantiza que el procesamiento siempre converge y que las dependencias entre etapas son explícitas: un nodo solo puede procesar datos cuando sus predecesores ya los produjeron.

3.1. UC1 - Filtro directo

UC1 establece el patrón base del sistema. El **Gateway** entrega el registro completo (**origin**, **destination**, **amount**, **format**) y el **Filter** aplica dos condiciones simultáneas: moneda USD y monto menor a \$50. Los registros que las satisfacen salen como (**origin**, **destination**, **amount**) hacia el **Join**. El campo **format** se descarta en esta etapa porque no aporta información al resultado.

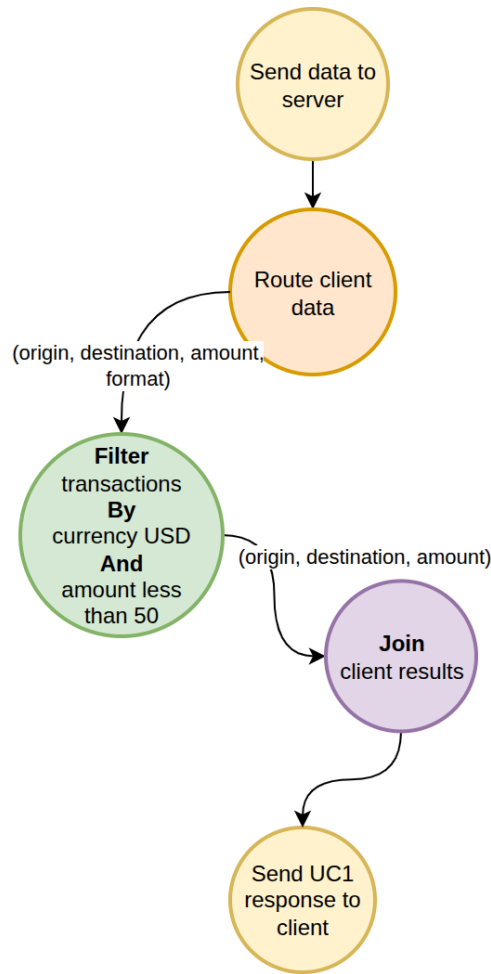


Figura 2: Diagrama DAG UC1

Cada transacción se evalúa de forma completamente independiente de las demás. No hay computación entre filas, por lo que el pipeline puede procesar los registros en cualquier orden y en paralelo sin afectar la correctitud del resultado.

3.2. UC2 - Máximo por banco

UC2 introduce dos conceptos nuevos respecto de UC1: múltiples fuentes de datos y agregación en dos etapas. Primero se distribuyen dos streams simultáneamente: registros de transacciones (`bank_id`, `origin`, `amount`) hacia el pipeline de filtrado, y registros de cuentas (`bank_id`, `bank_name`) hacia un **GroupBy** de resolución de nombres que elimina registros que duplican la información con el nombre del banco.

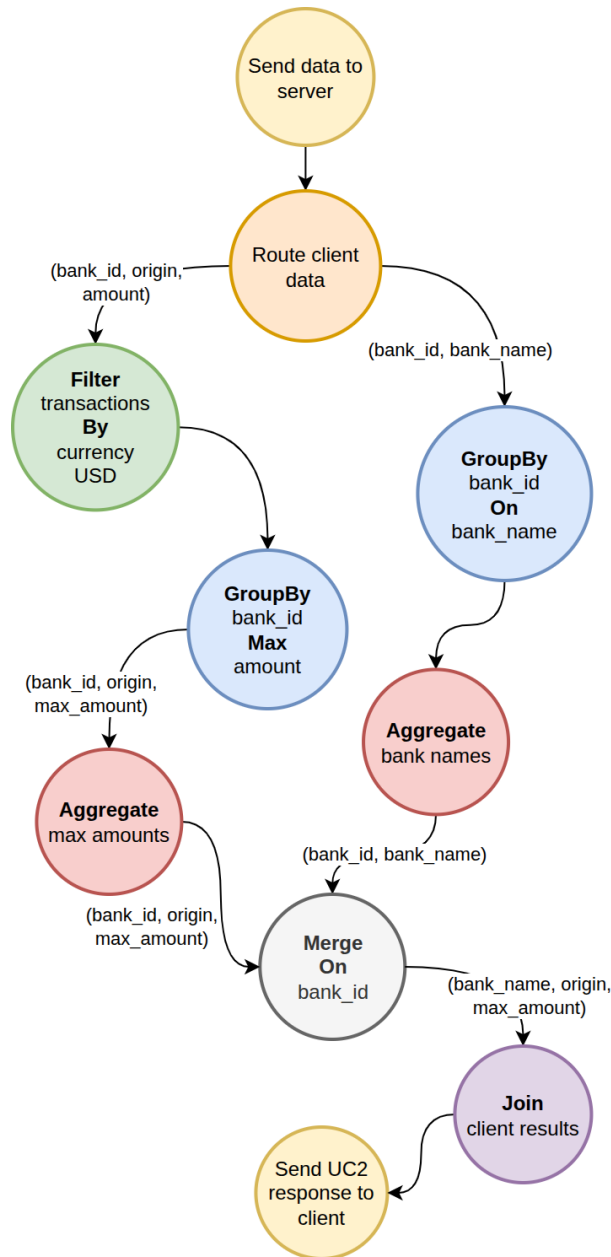


Figura 3: Diagrama DAG UC2

El pipeline de transacciones filtra por USD y luego el GroupBy calcula el máximo de **amount** por **bank_id** dentro de cada partición. El **Aggregate** consolida esos máximos parciales en el máximo global definitivo por banco.

3.3. UC3 - Comparación entre períodos

UC3 requiere comparar transacciones de dos períodos distintos: el umbral ($\text{amount} < \text{promedio_de_formato} / 100$) no es un valor fijo sino que depende del promedio global de cada formato en el período A, que solo se conoce una vez que se procesaron todos los datos de ese período.

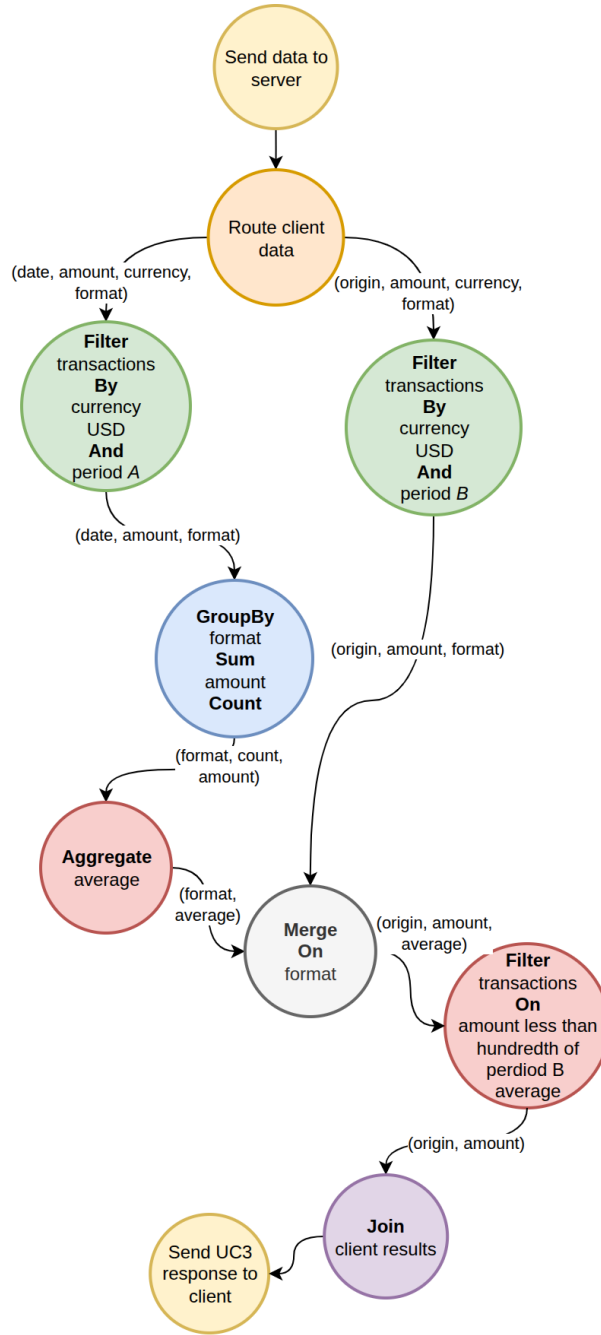


Figura 4: Diagrama DAG UC3

Primero se divide el stream de datos en dos bifurcaciones a ser procesadas en paralelo: una con currency USD y período A y la otra con currency USD y período B. El pipeline del período A materializa los promedios por formato: el GroupBy format Sum amount Count acumula suma y conteo por formato en cada partición, y el Aggregate computa el promedio real (sum/count). La salida es (format, average). El pipeline del período B produce el stream crudo de transacciones candidatas: (origin, amount, format). El Merge combina ambas salidas usando el formato como clave, de modo que cada transacción del período B queda asociada con el promedio de su formato en el período A. Un Filter final aplica la condición $\text{amount} < \text{average}/100$ y produce (origin, amount) para el Join.

3.4. UC4 - Patrón scatter-gather

UC4 requiere detectar el patrón *scatter-gather* en el grafo de transacciones: una cuenta origen dispersa fondos hacia múltiples intermediarios, que los concentran en una cuenta destino. Esta detección no puede hacerse registro a registro.

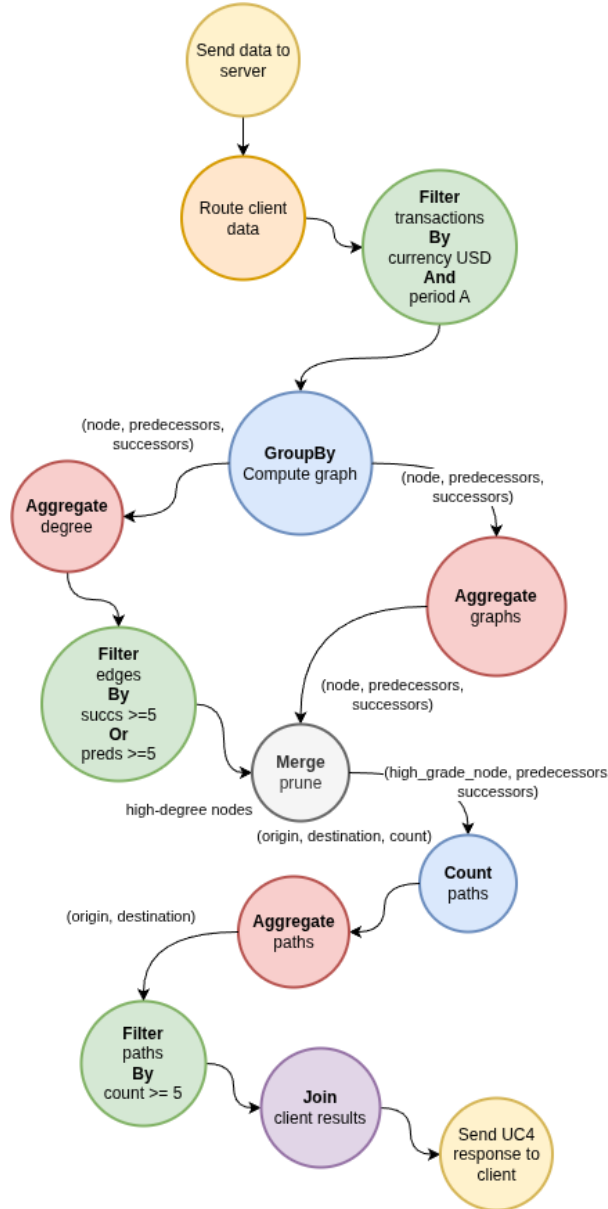


Figura 5: Diagrama DAG UC4

Después del filtro inicial (currency USD y período A), el **GroupBy Compute graph** construye la representación del grafo de transacciones: para cada nodo, el conjunto de predecesores y sucesores observados (**node, predecessors, successors**). Esta etapa shardea por **node**, de modo que todas las aristas de un mismo nodo se acumulan en la misma partición, y abre su salida (*fan-out*) hacia las dos ramas siguientes.

A partir de ese grafo, el constructor emite el mismo resultado hacia dos aggregators distintos:

- El **Aggregate degree** calcula el grado de entrada y de salida de cada nodo y produce dos conjuntos separados: los de **alto grado de salida** (al menos 5 sucesores,

candidatos a origen del patrón) y los de **alto grado de entrada** (al menos 5 predecesores, candidatos a destino). El umbral de 5 proviene del enunciado, que exige al menos 5 intermediarios. Un nodo que no entra en ninguno de los dos conjuntos no puede participar del patrón, por lo que esta rama actúa como una poda temprana que reduce el grafo antes del conteo costoso.

- El **Aggregate graphs** materializa el grafo completo (predecesores y sucesores consolidados por nodo).

El **Merge** (*prune*) cruza ambas salidas y conserva únicamente las aristas que van de un origen de alto grado de salida a un destino de alto grado de entrada.

Sobre ese grafo podado, el **Count paths** cuenta, para cada par (**origin**, **destination**), la cantidad de cuentas que sirven como intermediario directo entre ese origen y ese destino (es decir, el número de cuentas intermediarias del patrón scatter-gather), shardeando por par para repartir el conteo. El **Aggregate paths** consolida esos conteos parciales y conserva solo los pares con un conteo mayor o igual a 5. Por el volumen del grafo, las etapas que acumulan estado vuelcan sus estructuras parciales a disco (*spill*), particionadas en archivos por shard. El resultado (**origin**, **destination**) llega al Join.

3.5. UC5 - Conteo con conversión de moneda

UC5 cuenta cuántas transacciones de formato Wire o ACH en el período A tienen un monto menor a \$1 USD. Los montos pueden estar expresados en cualquier moneda, por lo que la comparación con el umbral solo es válida después de convertir.

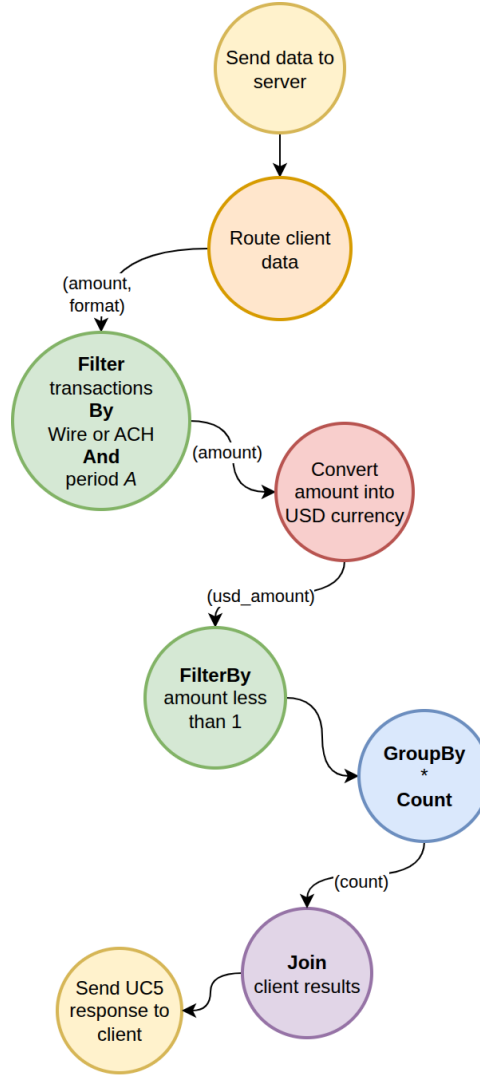


Figura 6: Diagrama DAG UC5

El Filter inicial aplica formato (Wire o ACH) y período A para reducir el dataset antes de la etapa de conversión. El paso de conversión traduce cada monto a su equivalente en dólares. El Filter siguiente aplica el umbral de \$1. Las transacciones por debajo de \$1 llegan al GroupBy * Count, donde el * indica que no hay clave de agrupación: todos los registros se colapsan en un único contador global. Ese conteo es el resultado final de UC5.

4. Vista física

4.1. Robustez

El diagrama de robustez muestra cómo se ejecutan los controladores en los pipelines: qué componentes concretos existen, qué colas los conectan, cuántas instancias corren en paralelo y qué estrategia de ruteo usa cada etapa.

Todo el ruteo entre etapas se resuelve por *afinidad*: cada enlace es un *exchange* directo de RabbitMQ con una cola por instancia downstream, ligada por una *routing key* igual a su índice de *shard*. El productor elige el shard de cada registro y publica con esa

clave, de modo que cada instancia consumidora es dueña exclusiva de su partición. No existen *working queues* con consumidores competitivos: cada cola tiene exactamente un consumidor.

El uso de colas por afinidad/shard en lugar de *working queues* competitivas hace que cada instancia sea dueña de una partición y facilita el razonamiento de recuperación ante fallos; su justificación se desarrolla en la sección de tolerancia a fallos.

La clave de *shard* depende de la etapa, y responde a uno de dos criterios:

- **Sharding por semántica del problema:** cuando todos los registros de una misma clave deben caer en la misma instancia para preservar la correctitud (por ejemplo, todos los del mismo `bank_id`, `format`, nodo del grafo o par del grafo). El shard se calcula sobre esa clave de dominio.
- **Sharding equitativo (pseudoaleatorio):** cuando no hay una clave de dominio que preservar y solo se quiere repartir carga entre réplicas. El productor reparte por la identidad del mensaje (un *hash* estable derivado del identificador del productor más su número de secuencia). Este criterio es el que reemplaza a las *working queues* competitivas.

Por el primer criterio, todas las instancias de `group_by` y `aggregate` fijan `PYTHONHASHSEED` para que la asignación de shards sea determinística y estable entre reinicios.

El último controlador, el **Join**, no se escala por datos sino que está **particionado por caso de uso**: recibe una configuración (`JOIN_PARTITION`) que indica qué casos de uso atiende cada réplica. Según la cantidad de réplicas de Join, se asignan uno o más UC a cada instancia para equilibrar la carga según el peso relativo de cada caso de uso, no según la cantidad de clientes. Así, por ejemplo, una réplica podría atender un único caso de uso y otra agrupar varios más livianos, según su peso relativo. Esta asignación no es automática: es configurable. La ventaja es que todas las respuestas parciales de un mismo cliente para un UC convergen en la misma réplica de Join, por lo que cada cola de UC tiene un único consumidor sin necesidad de shardear por `client_id`: la instancia dueña del UC reúne los resultados parciales de cada cliente y construye la respuesta final.

4.1.1. UC1

El `default_filter` corre con múltiples instancias en paralelo (`DEFAULT_FILTERS`), cada una procesando una partición del stream de transacciones que el Gateway reparte de forma equitativa entre ellas. Cada instancia aplica las dos condiciones (USD y monto menor a \$50) y, como UC1 no tiene etapas intermedias, deposita los registros que las satisfacen directamente en la cola de entrada del Join.

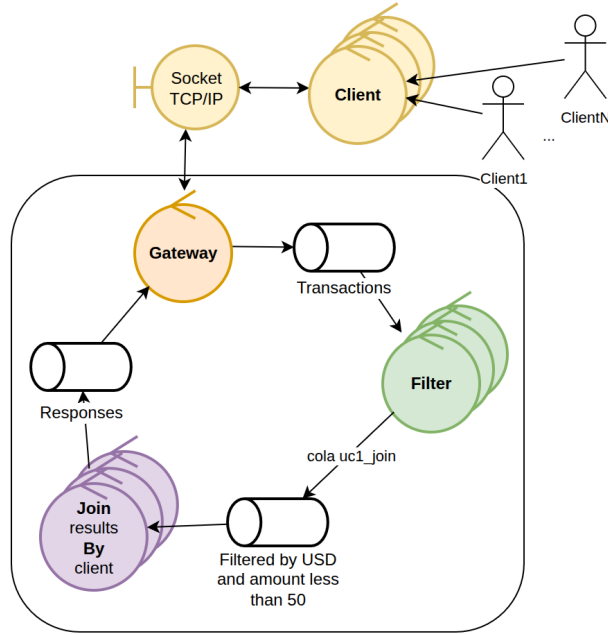


Figura 7: Diagrama de Robustez UC1

La cola de Responses desacopla el Join del Gateway: el Join deposita los resultados en la cola y el Gateway los consume para cerrar la conexión correspondiente.

Por el volumen de salida de UC1 (potencialmente millones de transacciones), el Join no acumula el resultado en memoria: lo vuelca a disco (*spill*) apenas lo recibe y lo emite hacia la cola de Responses en *batches* a medida que lo relee. El criterio acá es de volumen conocido y grande, por lo que se vuelca el total sin umbral.

4.1.2. UC2

El Gateway publica en dos exchanges distintos: **Transactions** para el stream de transacciones y **Accounts** para el dataset de cuentas. Los dos pipelines (el que resuelve el nombre de banco y el que obtiene su monto máximo) corren en paralelo. En el de transacciones, el **default_filter** filtra por USD, el **GroupBy bank_id Max amount** calcula el máximo parcial por **bank_id** en cada partición (con shardeo por **bank_id** entre **GroupBy** y **Aggregate**) y el **Aggregate max amounts** consolida el máximo global por banco. En el de cuentas, el **GroupBy bank_id On bank_name** extrae las asociaciones (**bank_id**, **bank_name**) y el **Aggregate bank names** las consolida.

El **Merge On bank_id** cruza ambos pipelines. Corre con varias instancias (**UC2_MERGES**) coordinadas en anillo. El **Aggregate** de máximos hace *broadcast* de su salida (cada instancia del Merge recibe los máximos de todos los bancos), mientras que el **Aggregate** de nombres shardea por **bank_id**, de modo que cada instancia del Merge es dueña de un subconjunto de bancos. Así, cada banco es emitido por exactamente una instancia del Merge: la que tiene su nombre en su shard. El resultado se deposita en la cola de entrada del Join.

Ninguna etapa de UC2 hace *spill*: el estado es chico (el máximo por **bank_id** y el mapa de nombres), por lo que se resuelve enteramente en memoria.

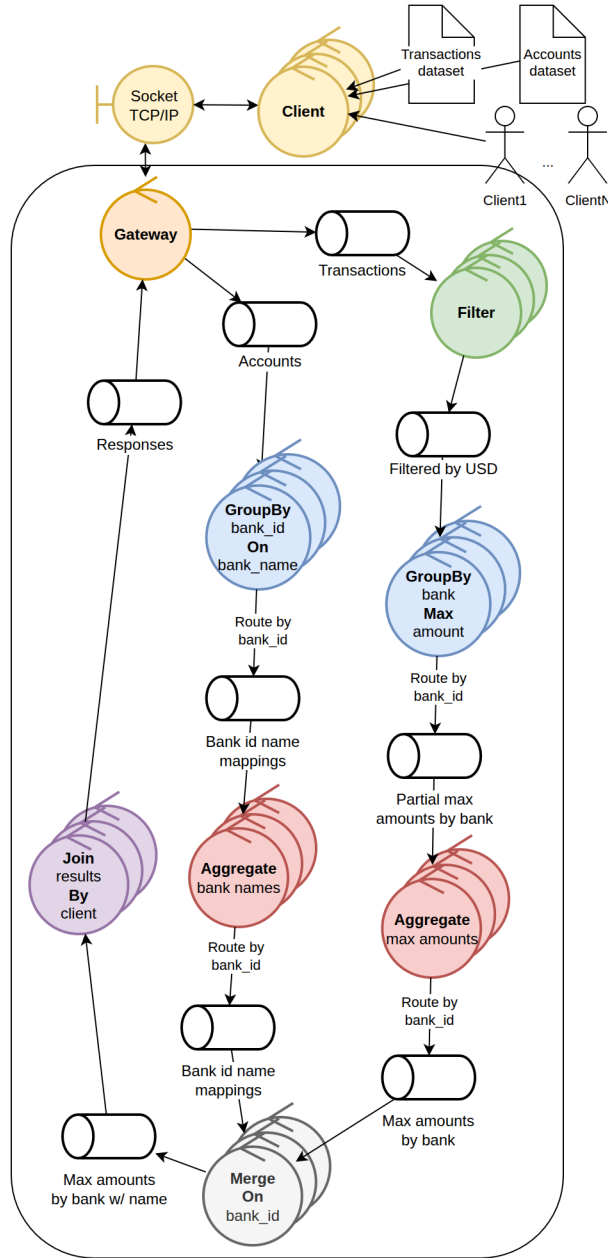


Figura 8: Diagrama de Robustez UC2

4.1.3. UC3

El `default_filter` produce dos exchanges de salida distintos: **Filtered by USD and period A** y **Filtered by USD and period B**. Además de filtrar por currency, rutea cada transacción al pipeline correspondiente según su fecha. Una transacción no puede pertenecer a ambos períodos, por lo que el ruteo es mutuamente exclusivo. El pipeline del período A usa shardeo por **format** entre el **GroupBy** y el **Aggregate** para que todos los registros del mismo formato lleguen a la misma instancia. Los promedios resultantes se distribuyen al **Merge** mediante *broadcast*: cada resultado del **Aggregate** se replica a todas las instancias del **Merge**, mientras que las transacciones del período B se reparten por shardeo entre esas mismas instancias. Cualquier instancia del **Merge** puede recibir transacciones del período B de cualquier formato y necesita los promedios de todos los formatos para evaluar la condición; el shardeo enviaría cada promedio a una sola ins-

tancia, mientras que el broadcast garantiza que todas tengan el contexto completo. El broadcast es aceptable dado que por la naturaleza de los datos del problema no puede haber demasiados formatos de pago; en particular, hay 7 formatos en el dataset de ejemplo entero.

Después del Merge, un segundo Filter aplica la condición `amount < average/100` y produce la cola final hacia el Join.

Por el volumen del período B (del orden de millones de transacciones), el **Merge** no mantiene ese lado en memoria: lo vuelca a disco (*spill*) apenas lo recibe y lo relee en *batches* al cruzarlo con los promedios; es el caso que de otro modo causaría un *out of memory*. El Join de UC3 también vuelca su salida a disco por el mismo motivo de volumen. En ambos el criterio es volcado total sin umbral.

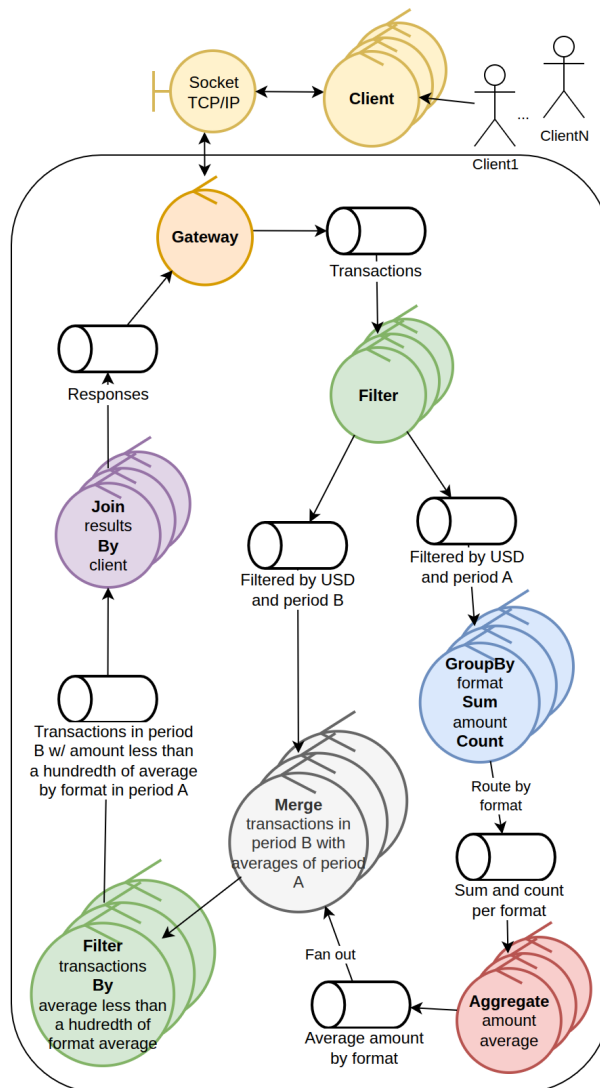


Figura 9: Diagrama de Robustez UC3

4.1.4. UC4

Tras el `default_filter` (USD y período A), un único `GroupBy Compute graph` construye la representación del grafo (ruteando por `node`) y abre su salida (*fan-out*) hacia dos `Aggregate` distintos. Esta etapa corre con varias instancias (`UC4_COMPUTE_GRAPHS`).

El primer Aggregate, **Aggregate degree** (`UC4_DEGREE_AGGREGATES`), selecciona los nodos de alto grado (al menos 5 predecesores y/o 5 sucesores) y los hace *broadcast* hacia todas las instancias del Merge. El segundo, **Aggregate graphs** (`UC4_AGGREGATE_GRAPHES`), materializa el grafo completo y lo shardea por **node** entre las instancias del Merge.

El **Merge prune** corre con varias instancias en anillo (`UC4_PRUNES`): cada una tiene los dos conjuntos de nodos de alto grado (por broadcast) y su shard del grafo (por sharding), y conserva solo las aristas de un origen de alto grado de salida a un destino de alto grado de entrada. Sobre ese grafo podado, el **Count paths** (`UC4_COUNT_PATHS`) cuenta los intermediarios por par (**origin, destination**) (shardeando por camino para repartir el conteo entre instancias) y el **Aggregate paths** (`UC4_PATHS_AGGREGATES`) consolida los conteos parciales, conservando los pares con conteo mayor o igual a 5 y depositándolos en la cola de entrada del Join.

Por el volumen del grafo, UC4 es el caso que más vuelca a disco (*spill*). Todas las etapas **Aggregate** (la materialización del grafo, el cálculo de grados, el conteo de caminos y la consolidación de conteos) vuelcan su estado parcial a archivos particionados por shard, y el **Merge prune** también vuelca su lado del grafo. Es necesario porque el grafo y, sobre todo, la explosión combinatoria de pares (**origin, destination**) en los nodos de alto grado generan estructuras que no entran en memoria. El criterio acá es por umbral: cada etapa acumula en memoria hasta un máximo (`MAX_AMOUNT`) y recién entonces vuelca, a diferencia del volcado total de UC3 y los Join.

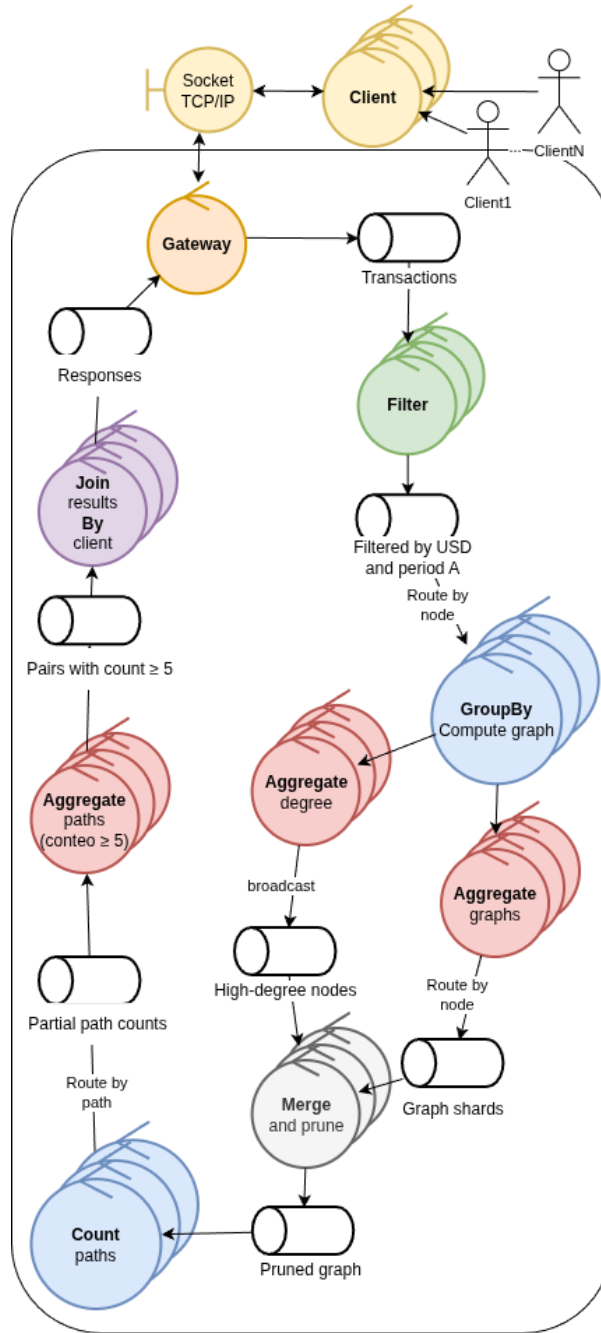


Figura 10: Diagrama de Robustez UC4

4.1.5. UC5

El controlador `Convert amount into USD` puede escalar horizontalmente de forma independiente del `Filter`. Se optó por permitir este escalamiento para más libertad en el despliegue del sistema, pero la operación que se realiza en esta instancia es puramente I/O intensive, y por tanto no requiere mayor poder de cómputo. Las cotizaciones se cachean por día: solo se emite una request al servicio externo de conversión ante un día todavía no visto, amortizando la latencia de red (en el período A son a lo sumo unos pocos días para toda la corrida). Tras la conversión, un segundo `Filter` aplica el umbral de \$1 USD y el `GroupBy * Count` produce un conteo parcial por instancia. Esos conteos parciales se depositan en la cola de entrada del `Join`, que los suma en el conteo global definitivo del

caso de uso.

Ninguna etapa de UC5 hace *spill*: el resultado es un conteo, por lo que el estado es chico y se mantiene en memoria.

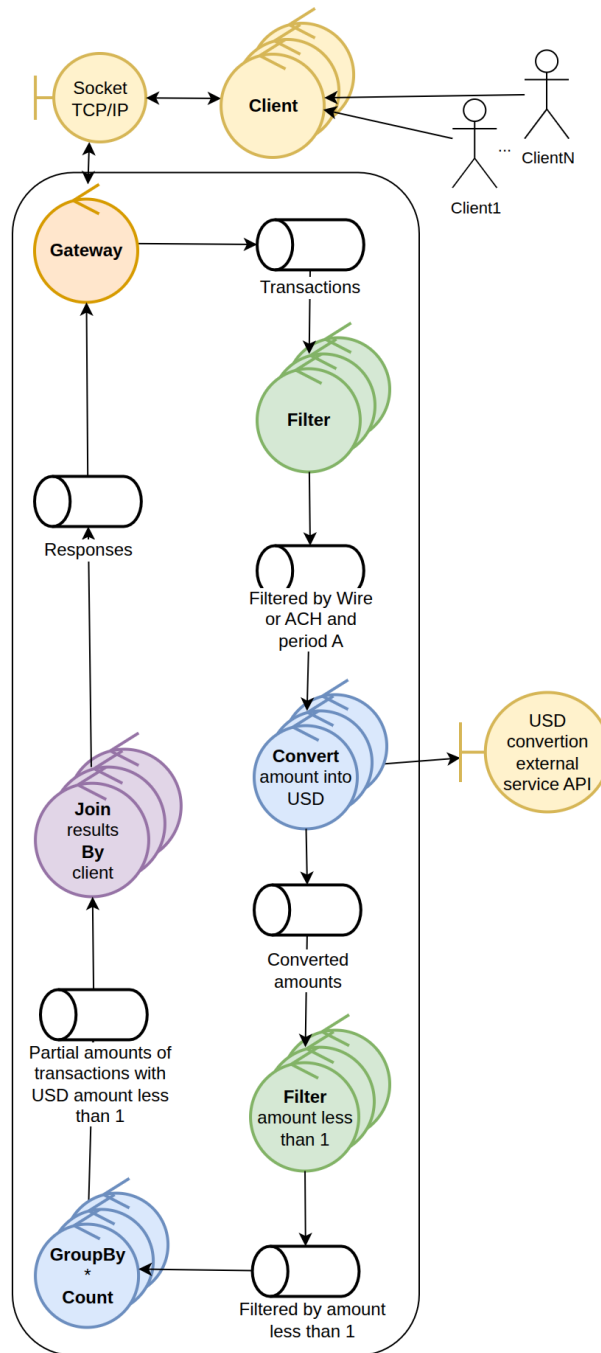


Figura 11: Diagrama de Robustez UC5

4.1.6. Overview del sistema completo

El sistema completo se organiza como una colección de pipelines independientes que comparten un único punto de entrada (el `default_filter`) y un único punto de salida (el `Join`).

El **Filter** inicial es el nodo con más conexiones: recibe el stream completo de transacciones y produce múltiples colas clasificadas por período, formato y moneda, una

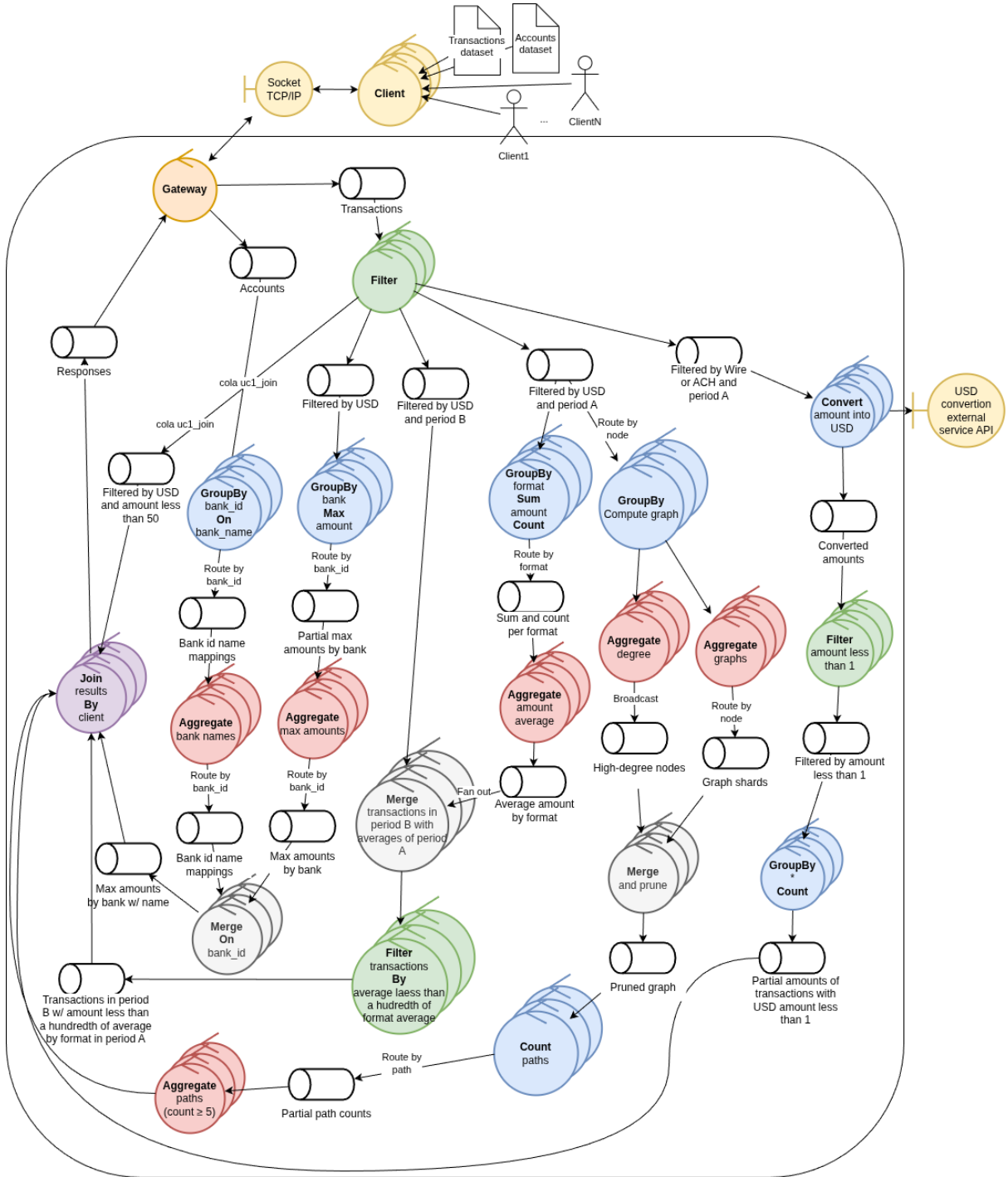


Figura 12: Diagrama de Robustez del sistema completo

topología *many-to-many* con los nodos downstream. Podría parecer un cuello de botella, pero no lo es, por dos razones. Primero, el filtrado es $O(1)$ por registro: descarta o rutea cada transacción con un costo mínimo, sin acumular estado ni tocar disco, de modo que difícilmente sea la etapa limitante. Segundo, si hiciera falta más capacidad, el Filter escala horizontalmente con más libertad que otros nodos: al ser *stateless*, se le pueden agregar instancias (DEFAULT_FILTERS) sin coordinación de estado.

Concentrar el filtrado inicial en un único tipo de controlador es, además, lo más simple. Varios casos de uso comparten condiciones iniciales (la mayoría filtra primero por

currency USD), así que un Filter común recorre cada *batch* una sola vez y rutea cada registro a las colas que correspondan. Los ruteos por clave hacia los **aggregate** y **merge** preservan la correctitud (todos los registros de un grupo caen en la misma instancia) y se aplican solo donde la cantidad de variantes es suficiente para seguir aprovechando la estrategia *multi-computing*.

Se consideraron dos alternativas para el filtrado, ambas descartadas:

- Encadenar el filtrado en varias instancias de Filter (por ejemplo, filtrar primero por currency USD y luego por monto menor a \$50). No reduce la cantidad de conexiones (la mayoría de los UC requieren el filtro de USD igual) y obliga a recorrer cada batch varias veces en vez de una sola.
- Segregar los filtros en distintas instancias según el filtro a aplicar. Es equivalente en robustez a escalar horizontalmente el único Filter, pero con más complejidad de ruteo.

4.2. Despliegue

Para el despliegue del sistema se intentó desacoplar el mismo por completo de la *vista lógica*: el hecho de que dos controladores realicen un filtrado, no debería impactar en la decisión de en qué nodos desplegarlos. Dicho esto, en el siguiente diagrama, se muestra cómo podrían desplegarse los procesos del pipeline de la manera más distribuida posible.

Cada controlador corre en su propio contenedor y se conecta al *broker* RabbitMQ, que actúa como middleware de comunicación por grupos. El **Gateway** es el borde entre el cliente y el sistema: recibe la conexión TCP del cliente, inyecta sus datos al pipeline y le devuelve las respuestas. Además de los nodos del pipeline, el despliegue incluye dos procesos de infraestructura: el contenedor de RabbitMQ y el **Supervisor**, un proceso de supervisión que recibe los *heartbeats* de todos los nodos, detecta los que dejan de responder y permite diagnosticar qué falló. Cada nodo con estado monta además un volumen propio donde persiste su *checkpoint*, de modo que al reiniciarse recupera su estado desde el mismo volumen. La detección de caídas se detalla en la sección de tolerancia a fallos; el mecanismo concreto que reinicia los contenedores y el **Chaos Monkey** que inyecta fallas pertenecen al entorno de simulación y se describen en Testing. Por ser el borde cliente-sistema, el Gateway se excluye de esas pruebas de Chaos Monkey (`CHAOS_EXCLUDE`): matarlo rompería la conexión TCP activa del cliente.

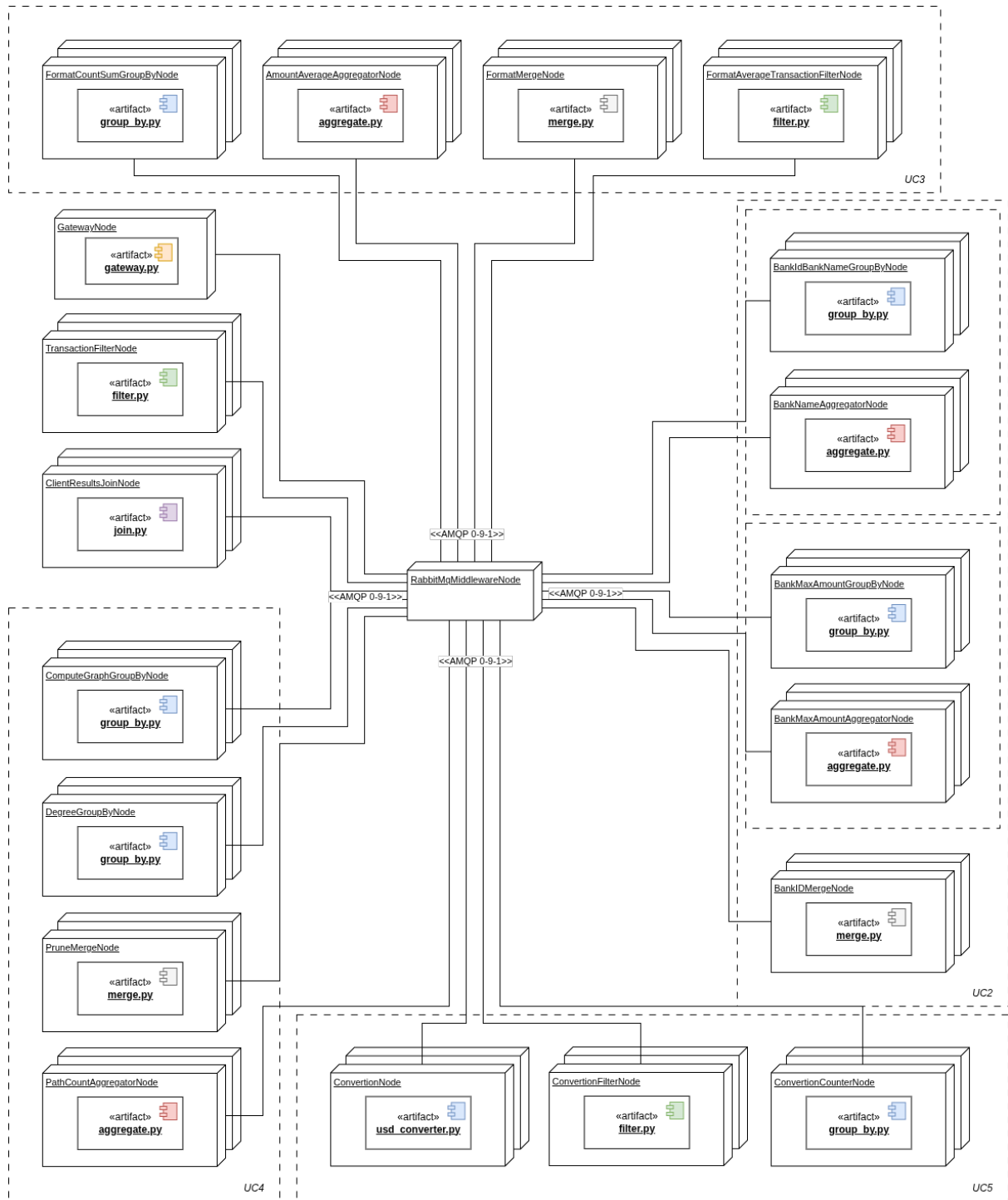


Figura 13: Diagrama de despliegue

5. Vista de desarrollo

5.1. Paquetes

Para la distribución de archivos fuente, se optó por mantener un directorio por cada *tipo* de controlador bajo `src/` (`filter/`, `group-by/`, `aggregate/`, `merge/`, `join/`, `gateway/` y `client/`), con su archivo `main.py` que levanta el proceso, su correspondiente `Dockerfile` y sus dependencias específicas. Las dependencias comunes se centralizan

en `common/`, que cada Dockerfile copia junto al código del controlador. Sus subpaquetes principales son: `comms/`, la comunicación, que agrupa el middleware sobre RabbitMQ (`middleware/`), el modelo de mensajes (`messages/`) y la maquinaria de fin de stream (`eof_handler/`); `data/`, el modelo de dominio (`Transaction` y `Account`); `checkpoint/`, persistencia y deduplicación; y `heartbeat/`, el cliente de *heartbeat* hacia el Supervisor.

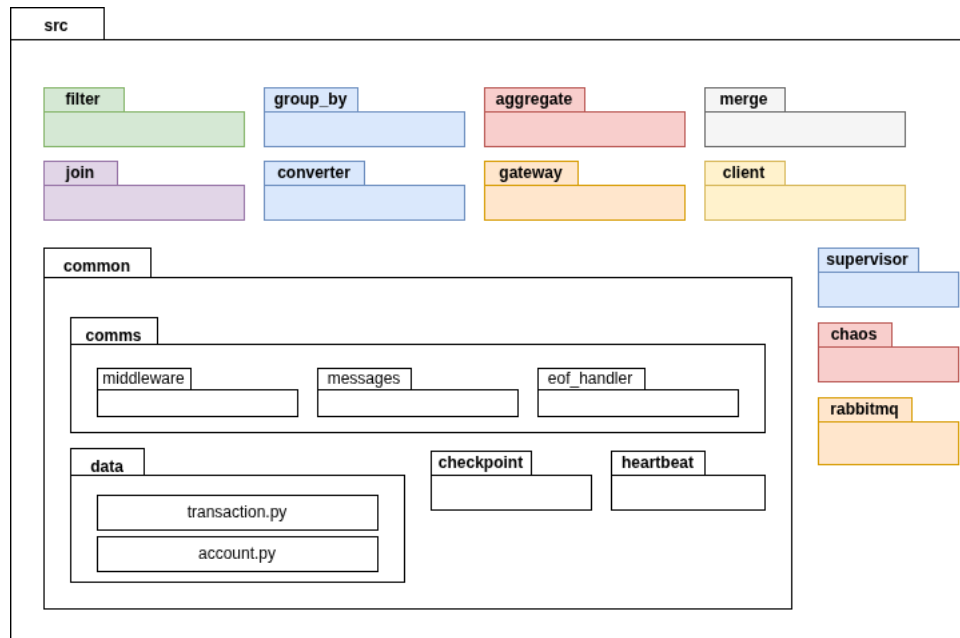


Figura 14: Diagrama de paquetes

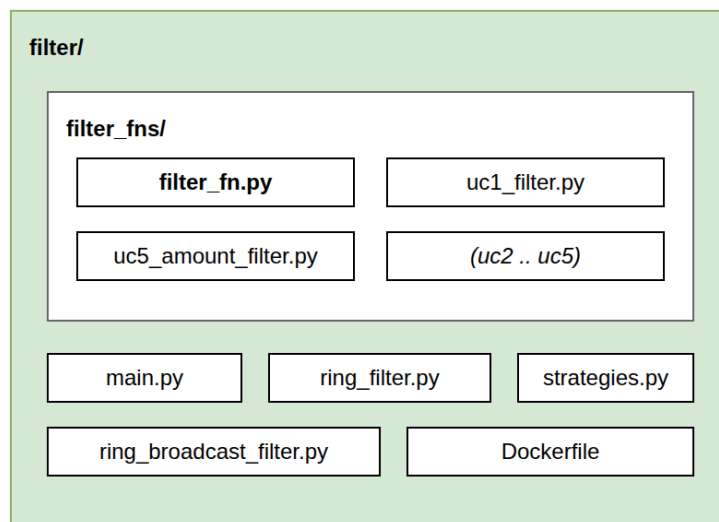


Figura 15: Diagrama de paquetes - `filter/`

La razón por la cuál se pueden generalizar los controladores es el patrón de implementación por el que se optó. Cada tipo de controlador tiene un nodo genérico (que hereda de `RingNode` (o de `StatelessRingNode` cuando no acumula estado) y resuelve el consumo, el ruteo por afinidad, la sincronización de fin de stream y el *checkpointing*) al que se le inyecta una *función de estrategia* que encapsula la lógica propia del caso de

uso. La estrategia concreta se elige al levantar el proceso mediante la variable de entorno `STRATEGY`.

Por ejemplo, el código fuente para un `Filter` consta del nodo genérico

```
class RingFilter(StatelessRingNode):
    def __init__(self, fn: FilterFn, ...):
        # ...
```

de la interfaz para la función de filtro

```
class FilterFn:
    @abstractmethod
    def filter(self, el: Message) -> Message:
        ...
```

y de la implementación particular del filtro que se quiera correr

```
class UC1Filter(FilterFn):
    def filter(self, el: Message) -> Message:
        # USD y monto < 50
        ...
```

El mismo esquema se repite para los demás controladores, cada uno con su propia interfaz de estrategia: `GroupByFn`, `AggregateFn`, `MergeFn` y `JoinFn`. Todas operan sobre la abstracción `Message`, de la que `Transaction` y `Account` son los datos de dominio que viaja en los mensajes de entrada.

6. Tolerancia a fallos

El sistema asume que cualquier nodo del pipeline puede caerse en cualquier momento y debe recuperarse manteniendo la correctitud de los resultados. El modelo de fallas considerado es la caída abrupta de un proceso; se asume que los nodos del Gateway y del *broker* RabbitMQ, junto con sus colas durables sobreviven; y que el volumen de estado de cada nodo persiste a través de los reinicios.

La garantía de correctitud se construye combinando dos mecanismos. RabbitMQ entrega **al menos una vez**: todo mensaje no confirmado (*ack*) se reenvía tras una caída. Sobre esa base, la **deduplicación** descarta los reenvíos ya procesados, de modo que el efecto observable es un procesamiento **efectivamente único**. La detección de los nodos caídos queda a cargo de un **Supervisor** externo, mientras que la consistencia del estado tras el reinicio la garantiza el **checkpointing**.

6.1. Arquitectura de tolerancia a fallos

Dos piezas sostienen la tolerancia a fallos, ortogonales al pipeline de procesamiento:

- El **Supervisor**, un proceso externo de supervisión que recibe los *heartbeats* de todos los nodos y detecta los que dejan de responder, permitiendo diagnosticar qué nodo falló.
- El **checkpoint** local de cada nodo con estado, que persiste en un volumen propio su estado de negocio, su tabla de deduplicación y sus contadores de secuencia de salida.

La estrategia de ruteo es, además, parte de la tolerancia a fallos: como se explicó en robustez, el sharding determinístico hace que cada mensaje reentregado por RabbitMQ vuelva siempre a la misma partición del mismo nodo lógico. Esto mantiene el estado local de cada réplica acotado a su propia partición y simplifica la deduplicación y la recuperación. Con *working queues* competitivas, en cambio, un reenvío podría caer en otra réplica, complicando el razonamiento sobre estado y duplicados; se optó por sharding determinístico precisamente para evitarlo, con una pérdida de performance empíricamente despreciable en los datasets evaluados (las etapas afectadas eran *stateless* y rápidas).

La **recuperación** de un nodo caído (reiniciar su proceso para que restaure su *checkpoint*, se reconecte a RabbitMQ y reanude el consumo) depende del entorno de ejecución. En la simulación con Docker un *reviver* reinicia el contenedor; ese mecanismo, sus variables de configuración y la línea de tiempo de recuperación se describen en Testing.

6.3. Cluster de supervisores

El supervisor es entonces el encargado de detectar cuando un nodo se cae, proveyendo así la posibilidad de volver a levantarlo. ¿Pero qué pasa si se cae el supervisor? Para resolver esto se agregaron réplicas del nodo supervisor, de manera tal que este último pasa de ser un único nodo a un *servicio tolerante a fallos*.

En el clúster de supervisores, el *líder* es el encargado de llevar la cuenta del estado de los nodos del pipeline. Por otro lado, para controlar el estado del líder, las réplicas del clúster le mandan *pings* con un delay configurable, a los cuales el líder contesta con *pongs* para certificar que sigue vivo. Cuando una réplica detecta que el líder se cayó, entonces se inicia una *elección de líder*. En particular, se implementó la *bully leader election* por ser más simple de programar.

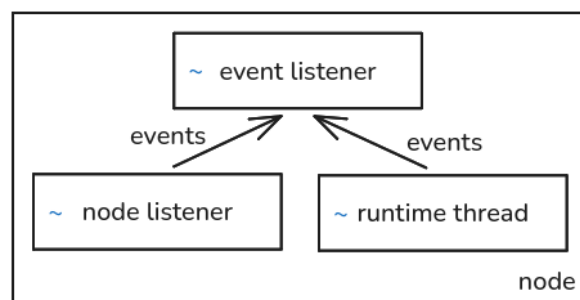


Figura 18: Clúster de supervisores y nodos del pipeline

La implementación de los supervisores consta principalmente de tres módulos:

- **Event loop.** Lee de la cola de eventos y handlea acordemente.
- **Runtime.** Lleva a cabo la funcionalidad del *rol* actual del supervisor (líder o réplica), implementado con un *strategy pattern*. Cuando se cae el líder, se pushea el evento *LeaderDown* en la cola de eventos.
- **Internal/node listener.** Acepta conexiones de otros nodos del clúster y las pushea en la cola de eventos.

Los eventos están implementados con una *multiple producer single consumer queue*: los productores son el runtime y el listener; y el consumidor es el event loop. El listener está basado en el *modelo de actores*, escucha mensajes y delega el manejo y la correspondiente contestación de los mismos al event loop.

Se optó por mantener conexiones TCP entre los nodos, para evitar casos borde de pérdida de mensajes en el clúster. Algo a notar de la implementación, es que busca minimizar la cantidad de conexiones activas entre nodos. Normalmente, cuando el líder está vivo, sólo existen $N - 1$ conexiones, con N la cantidad de nodos: las réplicas conectadas al líder para hacer el ping pong. Cuando se cae el líder, se inicializan sólo las conexiones

necesarias para llevar a cabo la bully leader election: la réplica que detecta la caída instantánea de conexiones con aquellos otros nodos con un ID mayor al suyo, manda mensaje de elección, espera el ACK, y corta la conexión; y así sucesivamente hasta que la cantidad de ACKs recibidos para el mensaje *election* es cero, en cuyo caso el nodo sabe que es aquel que sigue vivo con el mayor id del clúster, y broadcastea entonces el mensaje *coordinator*, volviendo así a las $N - 1$ conexiones de la normalidad.

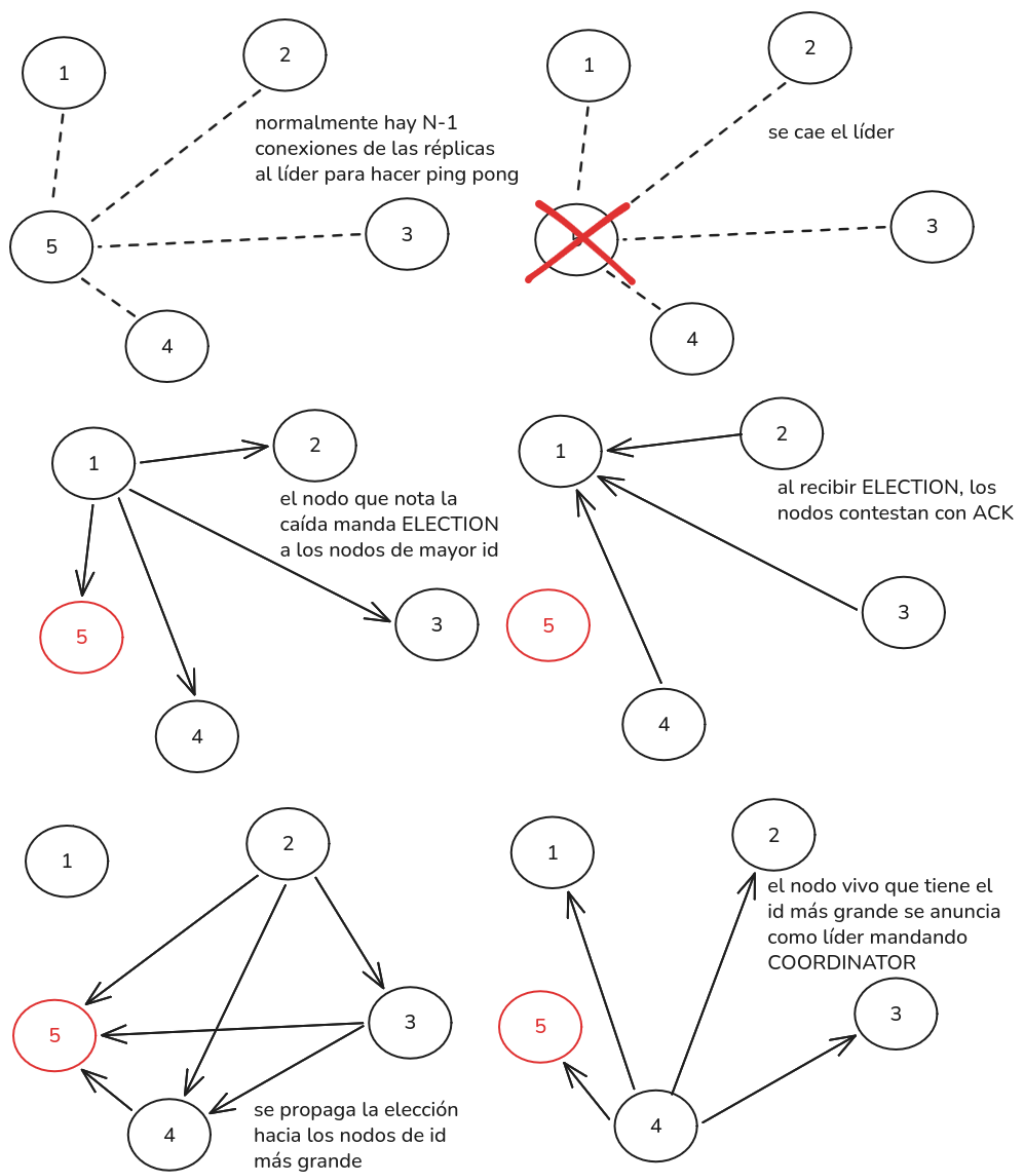


Figura 19: Bully leader election en el clúster de supervisores

6.4. Fin de stream y barrera de anillo

El fin de un stream se señala con un mensaje especial de *EOF*. Cuando el Gateway termina de repartir los datos de un cliente, emite un EOF por cada *shard* con la cantidad exacta de mensajes que envió a ese shard, de modo que cada instancia downstream sabe cuántos registros debe haber recibido antes de darse por completa.

El problema aparece cuando una etapa con N instancias en paralelo debe emitir un único EOF hacia la etapa siguiente: ninguna instancia, por sí sola, sabe si las demás

ya terminaron. Para resolverlo, las instancias de cada etapa se organizan en un **anillo** y ejecutan una barrera distribuida. Cuando una instancia alcanza su conteo esperado, vacía su estado acumulado hacia downstream y registra cuántos mensajes emitió por shard. Un *token* de barrera circula por el anillo acumulando los conteos de cada instancia; cuando vuelve al líder con los aportes de las N , éste emite hacia la etapa siguiente un EOF por shard con el total agregado del clúster. Así, la etapa downstream recibe exactamente la cuenta total que debe esperar.

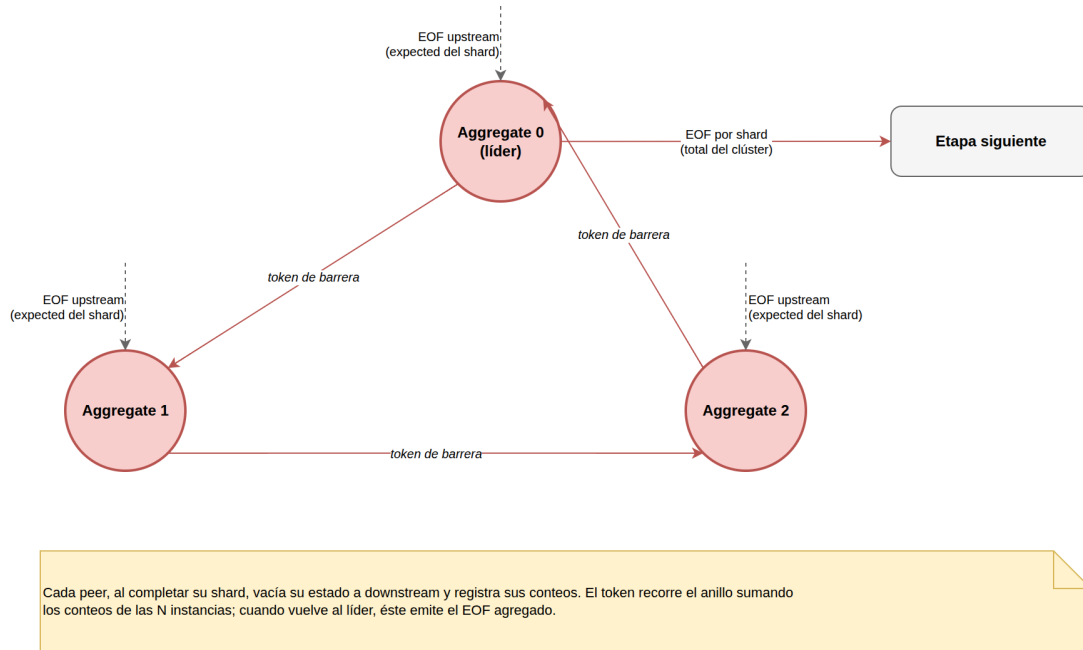


Figura 20: Propagación de EOF mediante la barrera de anillo

Este mecanismo es también el que indica a las etapas con estado (**aggregate**, **merge**) cuándo materializar su resultado: una instancia sólo vacía sus agregados parciales al completarse su entrada. El **merge** es un caso particular porque tiene dos entradas (la chica, por *broadcast*, y la grande, por *sharding*): sólo se da por completo cuando llegaron los EOF de ambas, usando la suma de los dos conteos. El Join, en cambio, es de instancia única por caso de uso: no participa del anillo, sino que espera el EOF agregado y, al alcanzar la cuenta, materializa y emite las respuestas del cliente.

La comunicación lógica de anillo se logra mediante el uso de un Exchange de RabbitMQ. Cada nodo suscribe a su ID y rutea sus mensajes al ID que le sigue (si hay 3, el 0 al 1, el 1 al 2 y el 2 al 0).

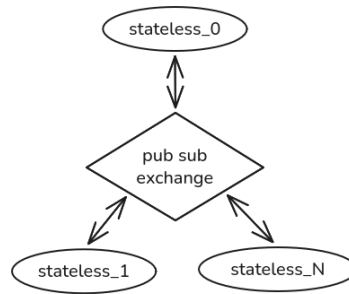


Figura 21: Infraestructura de la comunicación lógica en anillo

6.5. Checkpointing y deduplicación

Cada nodo con estado persiste periódicamente un *checkpoint* que incluye su estado de negocio, la tabla de deduplicación y los contadores de secuencia de salida. Para evitar escrituras incompletas: el bloque se vuelca primero a un archivo temporal y, sólo cuando la escritura terminó correctamente, se renombra sobre el checkpoint anterior. Como el *rename* es una operación atómica del sistema de archivos, nunca puede quedar un checkpoint **parcial o corrupto** como estado válido: si el nodo cae mientras escribe el temporal, el checkpoint anterior queda intacto y es el que se restaura.

La pieza clave es el **orden entre el checkpoint y los acks**. Los mensajes se procesan en batch (de tamaño `CHECKPOINT_EVERY`) y sus *acks* se retienen: recién se confirman a RabbitMQ **después** de que el *checkpoint* quedó escrito en disco. Así, todo lo confirmado tiene su estado ya persistido; si el nodo cae con *acks* pendientes, RabbitMQ reenvía esos mensajes y se reprocesan sobre el estado restaurado.

(MAX_AMOUNT) y recién ahí se vuelca. En las etapas cuyo volumen es conocido y siempre grande (el Merge de UC3 y los Join de UC1 y UC3), se vuelca el total a disco apenas llega, sin mantenerlo en memoria, y se relee en *batches*.

6.6. Puntos de falla

La combinación de entrega al-menos-una-vez, deduplicación, *acks* retenidos y barrera idempotente cubre los distintos momentos en que un nodo puede caer. A continuación se enumeran los puntos protegidos más importantes y cómo se preserva la correctitud en cada uno.

6.6.1. Caída a mitad de batch

Un nodo procesa parte de un batch (aplicando estado y registrando deduplicación en memoria) y cae antes de *checkpointear*. Como esos mensajes no se confirmaron, RabbitMQ los reenvía; el estado y la deduplicación se restauran del último *checkpoint* y los mensajes se reprocesan limpiamente sobre él.

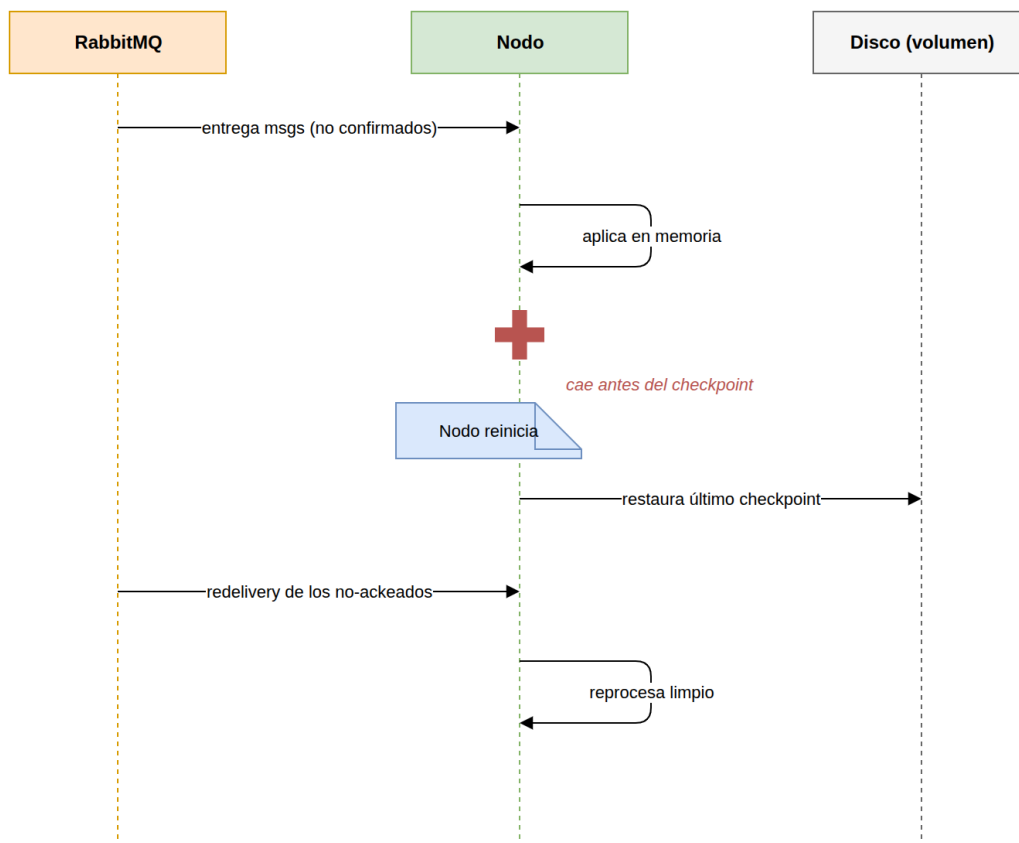


Figura 23: Caída a mitad de batch

6.6.2. Caída tras el checkpoint y antes de los acks

El nodo escribe el *checkpoint* del batch y cae antes de confirmar los *acks*. RabbitMQ reenvía el batch completo, pero al restaurar, la tabla de deduplicación ya contiene esas

secuencias: cada mensaje reenviado se detecta como duplicado, se confirma y se descarta sin volver a aplicarse. El estado queda correcto, tomado del *checkpoint*.

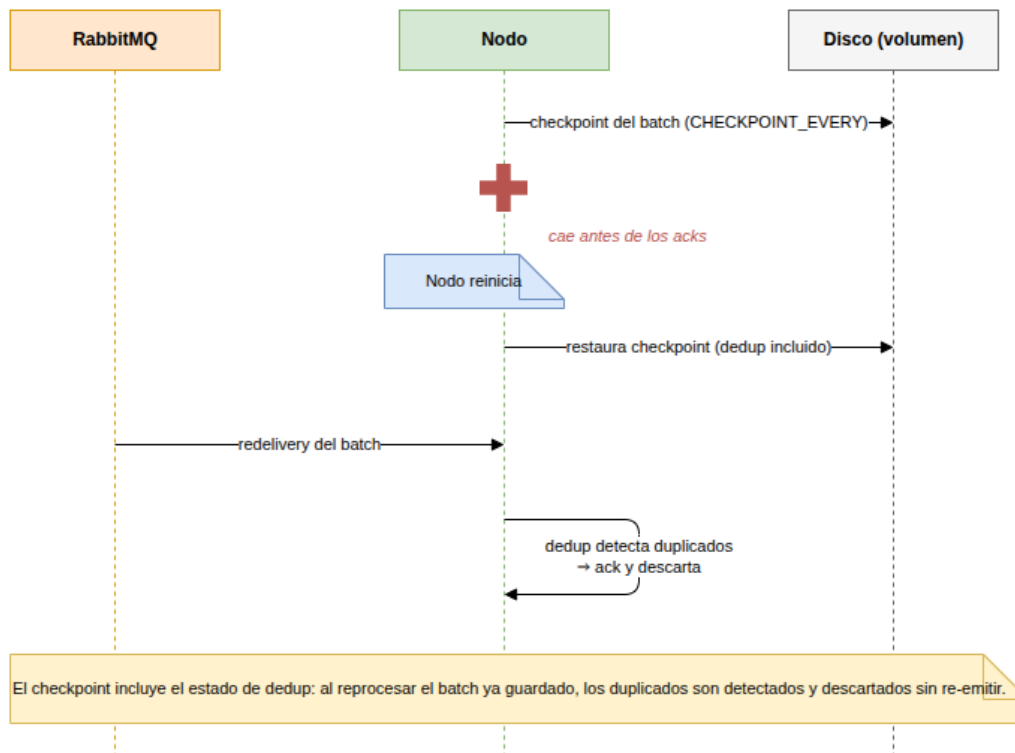


Figura 24: Caída tras el checkpoint, antes de los acks

6.6.3. Caída de un nodo con estado acumulado

Un *aggregate* o *merge* cae con resultados parciales acumulados a lo largo de varios *checkpoints*. Se restaura el estado del último *checkpoint* y, si ya había emitido y enviado el *token* de barrera, la barrera es idempotente: re-recibir el token sólo vuelve a registrar los mismos conteos, sin doble emisión. Los archivos de *spill* se truncan a su *offset* confirmado.

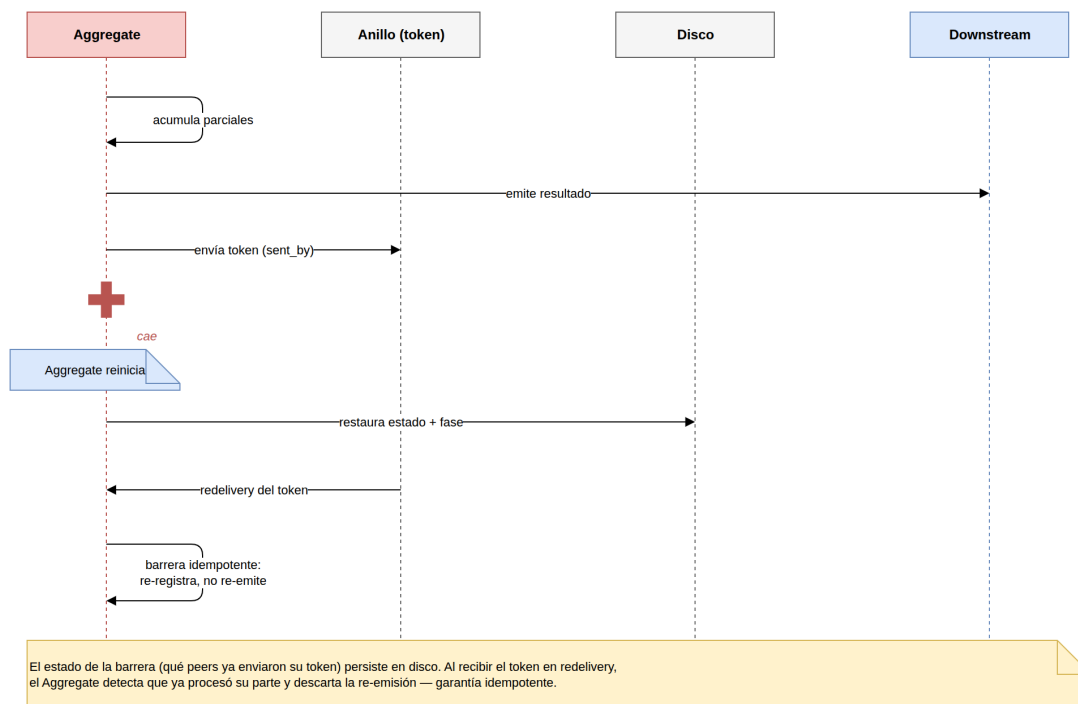


Figura 25: Caída de un nodo con estado acumulado

6.6.4. Caída del cliente

El cliente también puede caerse, y se distinguen dos casos según el momento.

Si el cliente cae **mientras envía datos**, el Gateway lo detecta porque la lectura del socket de ese cliente devuelve un cierre o error de conexión antes de recibir el *EOF* del stream. En ese caso el Gateway emite un mensaje **Abort** por cada *shard*, estampado con el `client_id` de esa sesión, que viaja por el pipeline igual que los datos. Cada etapa que lo recibe **descarta** el estado parcial de ese cliente: en el Join, el **Abort** dispara el **discard** del estado de negocio del cliente, el borrado de sus contadores de recepción y el marcado del cliente como abortado en el *checkpoint*. Así no quedan sesiones huérfanas ni etapas esperando indefinidamente un EOF que nunca va a llegar.

Si el cliente cae **cuando el sistema ya está procesando**, el pipeline puede seguir avanzando o limpiar su estado según el mecanismo de **Abort**, pero el cliente ya no va a recibir las respuestas: su conexión dejó de existir. La solución no es “recuperar” al cliente, sino **abortar/cancelar la sesión**, liberar los recursos asociados a ese `client_id` y evitar que el Join o el Gateway queden esperando o acumulando resultados para una conexión inexistente.

6.7. Rendimiento bajo tolerancia a fallos

La tolerancia a fallos no es gratis: cada nodo manda latidos, *checkpointea*, **publica cada mensaje con confirmación del broker** (así un envío en vuelo no se pierde) y, ante una caída, reprocesa desde el último *checkpoint*. Para medir ese costo se corrió un *benchmark* con una topología escalada (al menos dos instancias por anillo/etapa relevante, de modo que matar una réplica no detiene una etapa) sometida al *chaos monkey*, que se **siembra con una semilla fija** para que las corridas sean reproducibles por configuración.

Estos resultados deben leerse como **tendencias y conclusiones conceptuales**, no como valores absolutos. Aunque las corridas sean reproducibles por configuración/semilla, el *chaos monkey* introduce caídas sobre nodos y combinaciones que pueden variar entre ejecuciones, por lo que los valores exactos pueden cambiar. Por eso el análisis se enfoca en conclusiones estables sobre el comportamiento del sistema bajo fallos y no en números puntuales.

El estrés del *chaos* no es un solo número: el **kill-rate** es el producto de *cuántos* nodos se matan por oleada y *cada cuánto* se mata una oleada. Por eso el plano de estrés se recorre con **dos cortes ortogonales**, y cada uno expone una falla distinta:

- **Frecuencia:** se fija el tamaño de la oleada y se *acorta el intervalo* entre oleadas. Estresa la falla **acumulativa**: el supervisor se va quedando atrás a medida que pasa el tiempo.
- **Ráfaga:** se fija un intervalo generoso y se *agranda la oleada*. Estresa la falla por **simultaneidad**: una sola oleada se lleva muchos nodos de golpe, antes de que el supervisor alcance a reaccionar.

Ambos cortes se corren sobre los tres datasets. Como sus tamaños difieren en un orden de magnitud, los tiempos absolutos no comparten un mismo eje (el dataset chico quedaría aplastado contra el piso). Por eso las dos curvas se muestran en **ralentización relativa**: el tiempo de cada corrida dividido por el de su propia corrida *sin chaos*. Así los tres datasets se leen juntos, y la línea de referencia en $\times 1$ marca el “sin degradación”.

Como referencia de los tiempos absolutos, las corridas **base** (sin *chaos*, topología `min2`, *checkpoint* cada 1000) tardaron: **small** $\approx 2,9$ min, **medium** $\approx 10,4$ min y **large** $\approx 32,6$ min.

6.7.1. Frecuencia: matar más seguido

A intervalos más cortos hay más caídas que recuperar mientras la corrida avanza. La curva muestra que en casi todo el rango el costo es **marginal**: las tres líneas viajan pegadas a la referencia. El sobre costo recién se dispara en el **régimen más agresivo**, y lo hace de forma marcadamente **super-lineal** (una rampa que se empina al final). La lectura conceptual es que la frecuencia de fallos es un eje *barato* hasta cruzar un umbral, y a partir de ahí el trabajo de reposición empieza a competir en serio con el de procesamiento. Aun en el extremo, lo medido **siempre completa**: el sistema se *ralentiza*, no se traba.

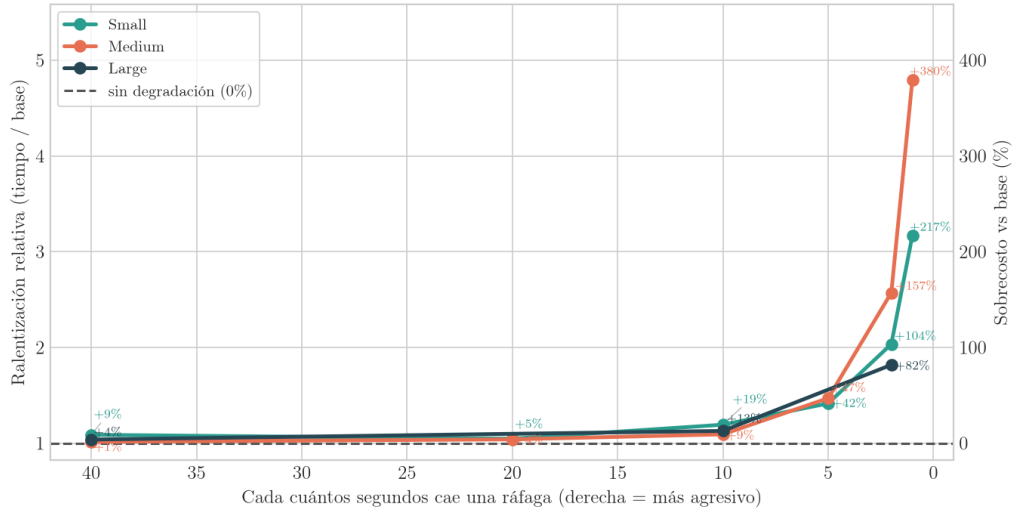


Figura 26: Frecuencia, ralentización relativa por dataset: la degradación es despreciable en casi todo el rango y sólo se empina en el régimen más agresivo.

6.7.2. Ráfaga: matar más de golpe

Con un intervalo holgado el supervisor tiene tiempo de sobra entre oleadas, así que lo que se mide es cuánta pérdida *instantánea* absorbe la redundancia de la topología escalada. El comportamiento es **cualitativamente distinto al de frecuencia**: la degradación crece de forma **gradual y sostenida** en todo el rango, sin rodilla ni salto abrupto. En el rango probado, la ráfaga degradó el tiempo pero no produjo colapso: eso muestra **robustez empírica bajo esas condiciones**, no una garantía universal para cualquier combinación de caídas. De combinar los dos cortes se desprende, conceptualmente, que el eje que más penaliza es la **acumulación de fallos en el tiempo** más que la simultaneidad puntual, al menos en el rango observado.

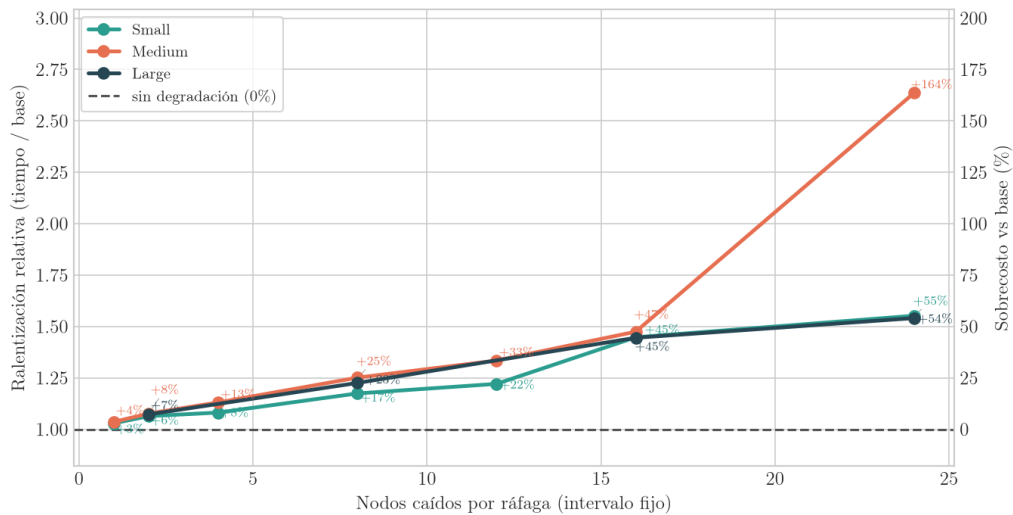


Figura 27: Ráfaga, ralentización relativa por dataset: degradación gradual y sostenida al crecer la oleada, sin colapso en ningún dataset.

6.7.3. El costo del checkpoint

La frecuencia de *checkpoint* traza una curva en **U** con dos costos en tensión. *Checkpointear* muy seguido está dominado por el costo de I/O; espaciar mucho lo abarata, pero encarece cada recuperación, porque ante una caída hay más mensajes para reprocesar. Sin fallos el óptimo es **amplio**: la curva *sin chaos* se aplanan en una meseta entre unos cientos y unos miles de mensajes por checkpoint.

Bajo fallos, en cambio, ese balance **se desplaza**. En la zona frecuente y en el centro de la meseta la curva *con chaos* corre casi pegada a la *sin chaos* (un sobre costo del orden del 5 al 25 %), pero al espaciar el checkpoint el sobre costo **crece de forma marcada**: en el punto más espaciado evaluado el chaos casi **duplica** el tiempo de la corrida base. Es el efecto teórico de que “espaciar encarece la recuperación” volviéndose visible: cuanto más raro es el checkpoint, más mensajes debe reprocesar un nodo revivido tras una caída, y ese reproceso pasa a dominar. La consecuencia práctica es que, bajo fallos, conviene un checkpoint **más frecuente** que el óptimo del caso normal, para mantener acotado el costo de recuperación.

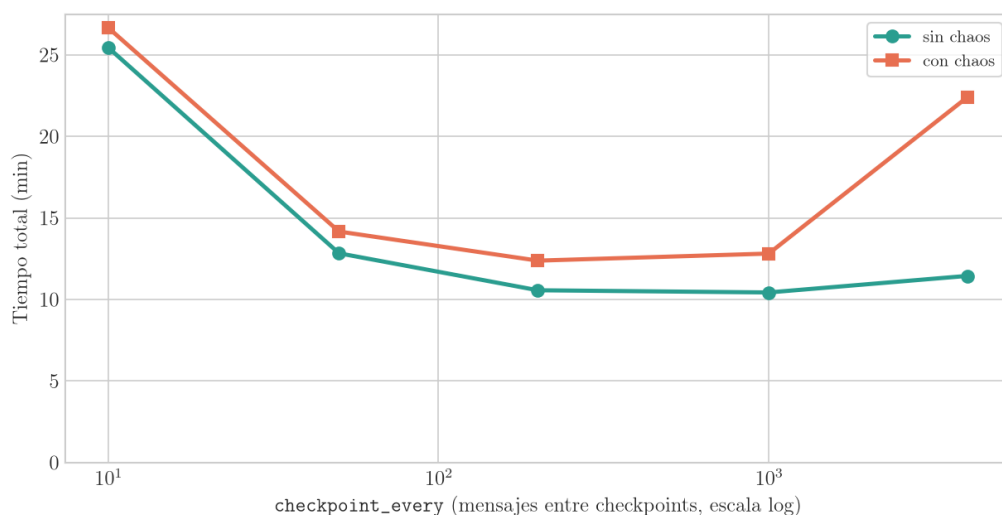


Figura 28: Costo del `CHECKPOINT_EVERY` (escala log): curva en U. Sin chaos el óptimo es amplio; bajo chaos el sobre costo es chico en la zona frecuente y media, pero crece al espaciar el checkpoint, casi duplicando el tiempo en el punto más espaciado evaluado.

7. Testing

El sistema se valida con dos tipos de pruebas: de **correctitud del pipeline** (que el resultado final de cada caso de uso sea el correcto) y de **tolerancia a fallos** (que esa correctitud se preserve ante caídas).

7.1. Testing del pipeline

Un script genera, de forma **secuencial**, los resultados esperados para un dataset dado (el *oráculo*). Al no ser distribuido puede ser más lento, pero por su simplicidad sirve como referencia de correctitud. Luego se levanta con Docker una simulación de todos los contenedores del sistema, se le envía el mismo dataset y se comparan los resultados finales de cada UC contra los esperados.

La prueba puede correrse:

- con un **único cliente** o con **múltiples clientes** simultáneos (`NCLIENTS`), para validar el aislamiento por `client_id`;
- con distintos **datasets**, seleccionables en `cfg.py`. El dataset `perfect` es chico, pero está construido a propósito para testear **correctitud**: incluye todos los casos borde posibles para los casos de uso específicos del enunciado. Los datasets `small`, `medium` y `large` son los datasets reales del problema, de tamaño creciente (~ 620 MB, ~ 2.8 GB y ~ 16 GB respectivamente), para validar el sistema a escala; son independientes los unos de los otros (los más chicos no son subsets del large).

El objetivo es comprobar que, para cada combinación, la salida de los cinco casos de uso coincida exactamente con la del oráculo.

7.2. Testing de tolerancia a fallos

Hay dos modos de probar la tolerancia a fallos.

7.2.1. Pruebas determinísticas

Se levanta el sistema con Docker y se inyectan caídas **controladas** en puntos críticos del pipeline (*crash points*): tras aplicar un batch pero antes de checkpointear, durante la escritura del checkpoint, antes de confirmar los *acks*, al propagar el EOF del anillo, al restaurar en el arranque, etc. Para cada nodo y cada punto de falla relevante se mata y se vuelve a levantar el nodo en ese momento exacto, y se verifica que el pipeline completo termine y que los resultados finales coincidan con el oráculo. Como recorrer la matriz completa “nodos $\times$ puntos de falla” es costoso, no siempre se ejecuta entera.

7.2.2. Demo con Chaos Monkey

El **Chaos Monkey** es un proceso de prueba que, mientras el sistema procesa, mata contenedores al azar para validar la recuperación de forma no determinística. Cuando un nodo cae, el **Supervisor** lo detecta por la ausencia de *heartbeats* y, en esta simulación, lo reinicia: los supervisores cumplen una función especial con **Docker-in-Docker** para poder levantar los contenedores caídos reutilizando su volumen de estado.

Es importante entender que esto es una **herramienta de prueba/demo**, no un supuesto de producción: en un sistema verdaderamente distribuido los nodos no serían simples contenedores locales reiniciables por Docker, sino procesos gestionados por el orquestador del entorno.

Las variables que gobiernan el Chaos Monkey y el mecanismo de supervisión/revival son configurables. Las principales:

- **CHAOS_ENABLED**: arma o desarma el Chaos Monkey; en 1 empieza a matar nodos.
- **CHAOS_INTERVAL**: cada cuánto mata nodos. Menor = fallas más frecuentes.
- **CHAOS_KILLS_MIN** / **CHAOS_KILLS_MAX**: cuántos nodos mata por ronda. Mayor = fallas más severas y simultáneas.
- **CHAOS_START_DELAY**: cuánto espera antes de empezar, para dar tiempo al *warmup* del sistema.
- **CHAOS_EXCLUDE**: nodos inmunes al caos (entre ellos el Gateway, RabbitMQ y el Supervisor); sacar uno lo vuelve vulnerable.
- **HEARTBEAT_INTERVAL**: cada cuánto late cada nodo. Menor = detección más rápida pero más tráfico.
- **HEARTBEAT_TIMEOUT**: cuánto sin latidos antes de marcar caído. Menor = detección más rápida pero más falsos positivos.
- **REVIVE_INTERVAL**: cada cuánto el reviver busca nodos caídos para reiniciarlos.
- *cooldown* de revival: espera mínima entre reintentos sobre un mismo nodo, para no martillar a uno que tarda en volver.

La línea de tiempo de recuperación de un nodo es, así, la suma de tres términos: el tiempo hasta perder el primer latido, el *timeout* de detección, y la espera hasta la siguiente pasada del reviver.

7.3. Testing de escalabilidad

Más allá de la correctitud con un cliente, se agregó una prueba e2e dedicada (**make scalability_test**, hermana de **make test_ft**) que verifica que los anillos de afinidad siguen produciendo el resultado correcto a medida que cada etapa se escala de un nodo a un anillo de 2 o 3. Para cada configuración de escalado (desde la topología base hasta topologías con todas las etapas en ≥ 2 instancias, o que escalan el camino caliente de UC4) reescribe los tamaños de los anillos, levanta el cluster y corre un cliente por cada dataset (perfect / small / medium / large), comparando el resultado contra el oráculo cacheado. Cada combinación puede repetirse varias veces para cubrir la variabilidad de *timing*. Ante una falla vuelca las colas trabadas, los códigos de salida de los nodos caídos y los *logs* alrededor, y al terminar verifica que el cluster, el estado en disco y las colas queden limpios.

8. Desarrollo

8.1. Tareas a realizar

Para el desarrollo del trabajo, se decidió que cada uno de los integrantes realice la implementación de uno o más casos de uso de punta a punta; con el objetivo de que todos podamos tener un seguimiento del total funcionamiento del sistema, pero sin la necesidad de entrar de lleno en los detalles específicos de cada caso de uso.

8.2. Asignación

La división del trabajo arrancó siguiendo una lógica **por caso de uso**. Las tareas de las que dependen todos los casos se implementaron primero entre todos los miembros del equipo y luego se fueron modificando según lo necesitara cada uno. El primer caso de uso se desarrolló en gran medida con la dinámica de *ping-pong pair-programming*, usando el caso más simple para que todo el equipo se interiorizara en paralelo de la implementación. El resto quedó dividido así:

- UC2: Lorenzo Minervino
- UC3: Federico Valsagna
- UC4: Alejo Ordoñez
- UC5: Lorenzo Minervino

A medida que el proyecto avanzó, sin embargo, el trabajo dejó de organizarse por caso de uso y pasó a estructurarse por **features transversales**, ya que la mayoría afectaba a muchas partes del sistema a la vez. Los refactors, el escalado de nodos, la tolerancia a fallos, el testing, los diagramas, la documentación y las correcciones se distribuyeron entre los integrantes a lo largo del proyecto, sin atarse a la asignación inicial por UC. Hubo, entonces, una asignación inicial por UC, pero la evolución real fue más *distribuida* (:P).